Práctica 5. Datos Anómalos

Análisis Exploratorio de Datos, Máster en Ciencia de Datos - UV

Contents

Introducción y objetivos **1**

Detección de inconsistencias en los datos **2**
- Ejercicio 1: Inspección Visual . . . 2
- Ejercicio 2: Valores especiales . . . 2
- Ejercicio 3: Valores perdidos y observaciones completas . . . 3

Detección de outliers univariante **4**
- Ejercicio 4: Estadísticos básicos . . . 4
- Ejercicio 5: Métodos de detección de outliers . . . 6
- Ejercicio 6: Detección de outliers . . . 6
- Ejercicio 7: Comparativa de los métodos de detección de outliers . . . 7
- Ejercicio 8: Representación gráfica . . . 8

Imputación de datos **9**
- Ejercicio 9: Imputación de datos perdidos con estadísticos básicos . . . 9
- Ejercicio 10: Representación gráfica . . . 10

Práctica 5 - Marino Fernández Pérez


```
library(pacman)
packages = c("knitr", "ggplot2","tidyr","dplyr","readr")
pacman::p_load(char=packages)
```

Introducción y objetivos

El objetivo de esta práctica es comprobar si los conjuntos de datos con los que vamos a trabajar son consistentes y, por tanto, pueden ser empleados para realizar un análisis sin errores. Las observaciones contenidas en los datos pueden presentar diferentes tipos de inconsistencias, como valores perdidos (missing values), valores especiales y valores anómalos (outliers).

Para evitar estas inconsistencias, estos datos deberán ser eliminados, corregidos o imputados. Siendo las etapas que pueden seguirse para obtener datos consistentes:

1. Detección de las inconsistencias.
2. Detectar el campo o campos que la generan y las dependencias entre ellos.
3. Corrección de errores.

Vamos a trabajar con el conjunto de datos *dirty_iris*, que ya conocemos, que contiene datos perdidos, valores especiales y anomalías de carácter numérico respecto a la distribución de los datos (`dirty_iris.csv`).

```
rm(list=ls())
dirty_iris <- read_csv("data/dirty_iris.csv")
```

Detección de inconsistencias en los datos

Por lo general, no hay un convenio o una representación común para indicar un dato perdido o incluso determinados valores especiales. En los siguientes ejercicios vamos a analizar cómo detectar inconsistencias presentes en los datos.

Ejercicio 1: Inspección Visual

Inspecciona visualmente los valores del conjunto de datos (`View`) y ordena de mayor a menor y de menor a mayor cada variable para descubrir las diferentes inconsistencias que presenta el conjunto `dirty_iris`. Comenta los valores que te llamen la atención.

Comenzamos utilizando la función `View()` para visualizar el dataset,

donde podemos ver datos perdidos `NA` y valores que parecen atípicos, al contar con medidas mucho mayores al resto (ejemplo: **petal.length = 63**).

Si clickamos en el nombre de cada una de las variables, se ordenan los valores recogidos de menor a mayor y viceversa. A simple vista, podemos observar que todas y cada una de las columnas cuentan con datos perdidos `NA` que deberán resolverse posteriormente.

Ordenando `Sepal.length` observamos valores que parecen alejarse bastante de la distribución de los datos así como **0.00*, que no tienen sentido ni aportan información ya que es impsoible que la longitud sea 0. Este mismo patrón se repite en las otras variables, aunque, además, en `Sepal.width`, hay un registro con valor negativo, que tampoco es representativo y no parece ser correcto.

```
View(dirty_iris)
```

Ejercicio 2: Valores especiales

Visualizar los datos puede ser muy revelador, pero en general, especialmente para conjuntos de datos grandes, realizaremos este análisis de forma automática.

Los valores especiales más comunes en R son:

- NA (*not available*): missing value o dato perdido (`is.na`)
- NULL (*empty set*): elemento nulo de longitud 0 (`is.null`)
- NaN (*not a number*): indeterminación con resultado desconocido (`is.nan`)
- Inf (*infinite*): valor numérico de valor infinito (`is.infinite`)

Nota: A parte de estas funciones, `is.numeric` comprueba si la variable es numérica; `is.finite` comprueba la existencia de cualquier valor especial en un vector numérico pero devuelve `FALSE` para variables de texto.

Reemplaza, en todas las columnas numéricas del data frame `dirty_iris`, los valores que sean especiales (usando `is.finite`), negativos, o iguales a cero, por `NA`. Nota: Puedes reemplazar los valores para cada variable del dataframe usando `mutate` con la función `ifelse`.

Siguiendo el enunciado, utilizamos `mutate()` junto con `across()` para aplicar `ifelse()` a todas las columnas numéricas del conjunto de datos.

Todos los valores que cumplen alguna de estas condiciones se sustituyen por NA, mientras que el resto se conserva sin modificaciones

La condición !is.finite(.) | . <= 0 permite identificar tanto los valores no finitos (NA, NaN, Inf, -Inf) como los negativos o iguales a cero. Estos últimos también se consideran inválidos, ya que las variables representan medidas físicas de una flor y, por tanto, deben ser estrictamente positivas.

Siguiendo el enunciado, utilizamos `mutate()` y, para seleccionar unicamente las var numéricas y aplicar la misma transformación en cada una de ellas, lo combinamos con `across(where(is.numeric))`. Así, se evita procesar cada columna por separado.

Por último, la transformación aplicada es identificar tanto los valores especiales indicados como los negativos o iguales a 0 por `NA`, utilizando `ifelse()` para evitar modificar aquellos que sí sean correctos.

```
# DEBEMOS ELIMINAR TAMBIEN LOS VALROES IGUALES A CERO YA QUE SE TRATA DE LONGITUDES DE PARTES DE UNA FL
dirty_iris <- dirty_iris %>% mutate(across(where(is.numeric), ~ ifelse(!is.finite(.) | . <= 0, NA, .)))
```

Para verificar la sustitución, utilizamos `summarise()` y `across()` para contar, en cada columna, los valores negativos o iguales a cero y los valores infinitos o `NaN`.

```
# comprobacion valores negativos ni ceros
dirty_iris %>% summarise(across(where(is.numeric), ~ sum(. <= 0, na.rm = TRUE)))
```

```
# A tibble: 1 x 4
  Sepal.Length Sepal.Width Petal.Length Petal.Width
         <int>       <int>        <int>       <int>
1            0           0            0           0
```

```
# comprobacion valores infinitos ni NaN
dirty_iris %>% summarise(across(where(is.numeric), ~ sum(is.infinite(.) | is.nan(.))))
```

```
# A tibble: 1 x 4
  Sepal.Length Sepal.Width Petal.Length Petal.Width
         <int>       <int>        <int>       <int>
1            0           0            0           0
```

Ambas comprobaciones devuelven **0 en todas las variables**, lo que confirma que se han sustituido correctamente los valores que no son váldosp por `NA`. No obstante, `NA` siguen siendo un valor perdido, y por tanto, valores que hacen que el dataset aún no sea consistente.

Ejercicio 3: Valores perdidos y observaciones completas

- Calcula el número y porcentaje de observaciones completas (muestras sin ningún dato perdido) empleando `complete.cases` (detecta todos los registros sin datos faltantes).

Como se indica en el enunciado, con `complete.cases` obtenemos un vector de booleanos que contiene solamente aquellas muestras que no tengan datos perdidos, es decir, `NA`. A partir del vector, con `sum` se suman los valores **TRUE** y devuelve el número de registros completos, y para calcular el porcentaje, se divide este valor entre el número total de muestras del dataset.

```
# num de observaciones
num_completas <- sum(complete.cases(dirty_iris))
num_completas
```

```
[1] 92
```

```
# porcentaje de observaciones
porc_completas <- num_completas / nrow(dirty_iris) * 100
porc_completas
```

```
[1] 61.33333
```

De ambos resultados, nos quedamos con el porcentaje, que indica que más de un 30 % de los datos contienen valores perdidos, principalmente porque todos lso valores erroneos se han sustituido por estos.

- Emplea después la función `na.omit` para quedarnos solo con las filas con datos completos en un nuevo data frame `clean_iris`.

Observando el manual de `na.omit`, vemos que devuelve un nuevo dataframe. Por tanto, el resultado de esta función lo asignamos al nuevo dataset limpio `clean_iris`.

```r
clean_iris <- na.omit(dirty_iris)
sum(is.na(clean_iris))
```

```
[1] 0
```

Tras limpiar el dataset, podemos ver que el numero de `NA` es 0.

```r
dim(clean_iris)
```

```
[1] 92  5
```

```r
dim(dirty_iris)
```

```
[1] 150   5
```

Detección de outliers univariante

Un outlier es una observación (o conjunto de observaciones) que no parecen consistentes con el resto de valores de esa variable. Por lo general, esas observaciones divergirán del patrón de comportamiento del resto de datos, es decir, presentarán una desviación respecto de la distribución de los datos.

En esta sección, vamos a centrarnos en métodos de detección univariantes, así que, en los ejercicios, a modo de ejemplo, analizaremos solo la variable `Petal.Length`.

Ejercicio 4: Estadísticos básicos

La presencia de outliers puede producir una gran influencia en caracterizaciones sencillas de los datos, como la media y la desviación típica.

Empleado la función `summarise y across`, calcula la media, la desviación típica, mediana, la *median absolute deviation from the median* (MADM) y el *interquartile range* (IQR) de las variables numéricas. Hazlo para el data frame con observaciones completas `clean_iris` (que no tiene NA pero tiene menos observaciones) y para el data frame con datos faltantes `dirty_iris` (que presenta muchos NA pero tiene todas las observaciones).

- Ten en cuenta que no se puede realizar un análisis estadístico con datos faltantes, pero en la mayoría de los casos se puede optar por no considerar dichos valores al realizar los cálculos (parámetro `na.rm=TRUE`).
- Para obtener el valor MADM, escribe una función `madm(x)` que sea robusta a la presencia de valores NA. Recuerda que el MADM se calcula como $1.4826 \cdot \text{median}(|x - \text{median}(x)|)$.
- Compara los resultados para los dos conjuntos en la variable `Petal.Length`.

Primero, definimos la función `madm()` tal y como se indica en el enunciado. Además, utilizamos `na.rm = TRUE` para evitar que los valores perdidos afecten al cálculo.

```r
madm <- function(x) {
  mediana <- median(x, na.rm = TRUE)
  1.4826 * median(abs(x - mediana), na.rm = TRUE)
}
```

Posteriormente, creamos una función generica `stats()` para calcular los estadisticos de ambos conjuntos utilizando `summarise()` y `across(where(is.numeric))`.

```r
stats <- function(data) {
  data %>% summarise(across(.cols = where(is.numeric), .fns = list(
                    media = ~ mean(., na.rm = TRUE),
                    sd = ~ sd(., na.rm = TRUE),
                    mediana = ~ median(., na.rm = TRUE),
                    madm = ~ madm(.),
                    IQR = ~ IQR(., na.rm = TRUE)
                  )))
```

```
}

stats_clean <- stats(clean_iris)
stats_dirty <- stats(dirty_iris)

stats_clean
```

```
# A tibble: 1 x 20
  Sepal.Length_media Sepal.Length_sd Sepal.Length_mediana Sepal.Length_madm Sepal.Length_IQR
               <dbl>           <dbl>                <dbl>             <dbl>            <dbl>
1               6.37            4.57                  5.8              1.04              1.3
# i 15 more variables: Sepal.Width_media <dbl>, Sepal.Width_sd <dbl>, Sepal.Width_mediana <dbl>,
#   Sepal.Width_madm <dbl>, Sepal.Width_IQR <dbl>, Petal.Length_media <dbl>, Petal.Length_sd <dbl>,
#   Petal.Length_mediana <dbl>, Petal.Length_madm <dbl>, Petal.Length_IQR <dbl>,
#   Petal.Width_media <dbl>, Petal.Width_sd <dbl>, Petal.Width_mediana <dbl>,
#   Petal.Width_madm <dbl>, Petal.Width_IQR <dbl>
```

```
stats_dirty
```

```
# A tibble: 1 x 20
  Sepal.Length_media Sepal.Length_sd Sepal.Length_mediana Sepal.Length_madm Sepal.Length_IQR
               <dbl>           <dbl>                <dbl>             <dbl>            <dbl>
1               6.61            6.80                  5.8              1.04              1.3
# i 15 more variables: Sepal.Width_media <dbl>, Sepal.Width_sd <dbl>, Sepal.Width_mediana <dbl>,
#   Sepal.Width_madm <dbl>, Sepal.Width_IQR <dbl>, Petal.Length_media <dbl>, Petal.Length_sd <dbl>,
#   Petal.Length_mediana <dbl>, Petal.Length_madm <dbl>, Petal.Length_IQR <dbl>,
#   Petal.Width_media <dbl>, Petal.Width_sd <dbl>, Petal.Width_mediana <dbl>,
#   Petal.Width_madm <dbl>, Petal.Width_IQR <dbl>
```

Al haber calculado 5 medidas estadísticas para cada una de las cuatro variables numéricas del dataset, los nuevos data frames `stats_clean` y `stats_dirty` contienen 20 columnas cada uno. Por tanto, para comparar los resultados de `Petal.Length`, seleccionamos las cinco columnas correspondientes a sus estadísticos utilizando `select()` e identificandolas como aquellas cuyos nombres comiencen por el de la variable original mediante `starts_with("Petal.Length")`.

```
stats_clean %>% select(starts_with("Petal.Length"))
```

```
# A tibble: 1 x 5
  Petal.Length_media Petal.Length_sd Petal.Length_mediana Petal.Length_madm Petal.Length_IQR
               <dbl>           <dbl>                <dbl>             <dbl>            <dbl>
1               4.16            2.88                  4.5              1.63              3.6
```

```
stats_dirty %>% select(starts_with("Petal.Length"))
```

```
# A tibble: 1 x 5
  Petal.Length_media Petal.Length_sd Petal.Length_mediana Petal.Length_madm Petal.Length_IQR
               <dbl>           <dbl>                <dbl>             <dbl>            <dbl>
1               4.48            5.78                  4.5              1.63              3.5
```

Los resultados obtenidos muestran que, aunque la media, la desviación y el IQR, son distintos para ambos, la mediana y el MADM se mantienen idénticos. En especial, estas dos ultimas son medidas que, teoricamente, son más robustas y menos sensibles a valores atípicos, que aún siguen estando presentes en nuestro dataset.

Por otro lado, podemos decir que la media y la desviación típica son más sensibles a la presencia de outliers.

Ejercicio 5: Métodos de detección de outliers

Escribe una función que detecte los outliers de un vector x para cada uno de los siguientes métodos de detección de outliers univariantes (todas las funciones tienen que ser robustas a la presencia de valores NA).

- **Regla 3 sigma** `reglasigma(x)`: Considera que los datos siguen una distribución gaussiana. Datos típicos caracterizados por la media y la dispersión por la varianza. Valores por encima de 3 desviaciones típicas se consideran outliers.
- **Identificador Hampel** `reglahampel(x)`: Utiliza estimadores robustos y no presupone que los datos siguen una distribución gaussiana. Datos típicos caracterizados por la mediana y la dispersión por la *median absolute deviation from the median* (MADM). Valores por encima de 3 MADM se consideran outliers.
- **Regla boxplot** `reglaboxplot(x)`: A partir de los datos se calcula su diagrama de caja (boxplot) y se definen como datos típicos superiores hasta el cuartil 75%, x_s; datos típicos inferiores hasta el cuartil 25%, x_i; y la dispersión se define como el rango intercuartílico ($IQR = x_s - x_i$). Valores por encima de $x_s + 1.5 \cdot IQR$ o por debajo de $x_i - 1.5 \cdot IQR$ se consideran outliers. Usa la función `quantile` para calcular los cuartiles.
- **Percentiles** `reglapercentil(x)`: Cualquier valor que esté fuera del rango del percentil 5% y 95% se puede considerar como atípico. Usa la función `quantile` para calcular los percentiles.

Siguiendo la definición de cada uno de los metodos de detección de outliers indicados en el enunciado, construimos las funciones utilizando la eliminación de valores `NA` para los distintos cálculos (`na.rm = TRUE`), evitando así obtener resultados iguales a `NA`.

Ejercicio 6: Detección de outliers

Utiliza la función `mutate` y `across` para identificar los outliers en la variable `Petal.Length` con todos los métodos de detección de outliers implementados. Genera una columna nueva para cada método. Nota: podría extenderse la aplicación a todas las variables numéricas usando también `mutate`, `across` y `where`.

Como no se indica nada a cerca de qué dataset utilizar, procesamos `dirty_iris` ya que en `clean_iris` no tenemos valores perdidos y, además, podriamos estar modificandolo incorrectamente y, por tanto, impidiendo el acceso a dichos datos para su recuperacion posteriormente.

Con `mutate()` y `across()` aplicamos los 4 métodos sobre la variable `Petal.Length` y generamos una columna nueva para cada uno de ellos. Como las funciones devuelven `TRUE` o `FALSE` según si una observación se considera atípica o no, podemos identificar fácilmente los outliers detectados por cada método.

```
dirty_iris <- dirty_iris %>% mutate(across(.cols = Petal.Length, .fns = list(
      sigma = ~ reglasigma(.),
      hampel = ~ reglahampel(.),
      boxplot = ~ reglaboxplot(.),
      percentil = ~ reglapercentil(.)
    )
  ))

select(dirty_iris, Petal.Length, starts_with("Petal.Length"))
```

```
# A tibble: 150 x 5
   Petal.Length Petal.Length_sigma Petal.Length_hampel Petal.Length_boxplot Petal.Length_percentil
          <dbl> <lgl>              <lgl>               <lgl>                 <lgl>
 1          4.5 FALSE              FALSE               FALSE                 FALSE
 2          6   FALSE              FALSE               FALSE                 FALSE
 3          5.4 FALSE              FALSE               FALSE                 FALSE
 4          1.6 FALSE              FALSE               FALSE                 FALSE
 5          3.5 FALSE              FALSE               FALSE                 FALSE
```

```
 6          NA   NA                 NA                  NA                  NA
 7           5.3 FALSE              FALSE               FALSE               FALSE
 8           5.1 FALSE              FALSE               FALSE               FALSE
 9           4.1 FALSE              FALSE               FALSE               FALSE
10           1.6 FALSE              FALSE               FALSE               FALSE
# i 140 more rows
```

Además, observamos que sigue habiendo valores `NA`, ya que, las funciones creadas no los tienen en cuenta para los calculos pero no los eliminan.

Ejercicio 7: Comparativa de los métodos de detección de outliers

En los métodos de detección de outliers analizados, examina los resultados comparando:

- Número de outliers detectados por método.

Para comparar los cuatro métodos, contamos los valores `TRUE` de las columnas del ejercicio anterior. Para ello, utilizamos `summarise()` sin considerar los valores `NA`.

```r
dirty_iris %>% summarise(across(starts_with("Petal.Length_"), ~ sum(., na.rm = TRUE)))
```

```
# A tibble: 1 x 4
  Petal.Length_sigma Petal.Length_hampel Petal.Length_boxplot Petal.Length_percentil
               <int>               <int>                <int>                  <int>
1                  2                   3                    3                     11
```

A partir de los resultados, podemos confirmar que, tal y como dice la teoria, la regla **3 sigma** es la menos agresiva y la regla de los **percentiles** la más fuerte.

- Los valores etiquetados como outliers (ordenados de menor a mayor).

Primero, utilizamos `pivot_longer()` para juntar las cuatro nuevas columnas en una sola. Después, con `filter()` nos quedamos únicamente con los valores identificados como outliers y con `group_by()` los agrupamos según el método.

Finalmente, con `summarise()` contamos los outliers con `n()` y mostramos sus valores ordenados de menor a mayor mediante `sort()`.

```r
dirty_iris %>%
  select(Petal.Length, starts_with("Petal.Length_")) %>%
  pivot_longer(
    cols = starts_with("Petal.Length_"),
    names_to = "Metodo",
    values_to = "Outlier"
  ) %>%
  filter(Outlier == TRUE) %>%
  group_by(Metodo) %>%
  summarise(
    Num_Outliers = n(),
    Valores = paste(sort(Petal.Length), collapse = ", ")
  )
```

```
# A tibble: 4 x 3
  Metodo                Num_Outliers Valores
  <chr>                        <int> <chr>
1 Petal.Length_boxplot             3 14, 23, 63
2 Petal.Length_hampel              3 14, 23, 63
3 Petal.Length_percentil          11 0.82, 0.925, 1.1, 1.2, 6.6, 6.7, 6.7, 6.9, 14, 23, 63
4 Petal.Length_sigma               2 23, 63
```

A partir de los resultados, observamos que **Hampel y boxplot coinciden** tanto en el número de outliers como en los valores considerados como atípicos. Con la regla de las **3 sigma** se logra identificar los valores `23` y `63`, que coinciden con dos de los registrados por los métodos anteriores, pero no el otro restante, posiblemente por ser menos agresiva que las otras dos técnicas.

Por otro lado, el método de percentiles detecta **11** outliers, incluyendo valores bastante menos extremos (`0.82` o `0.925`). Esto se debe al criterio que utiliza para discriminar atípicos.

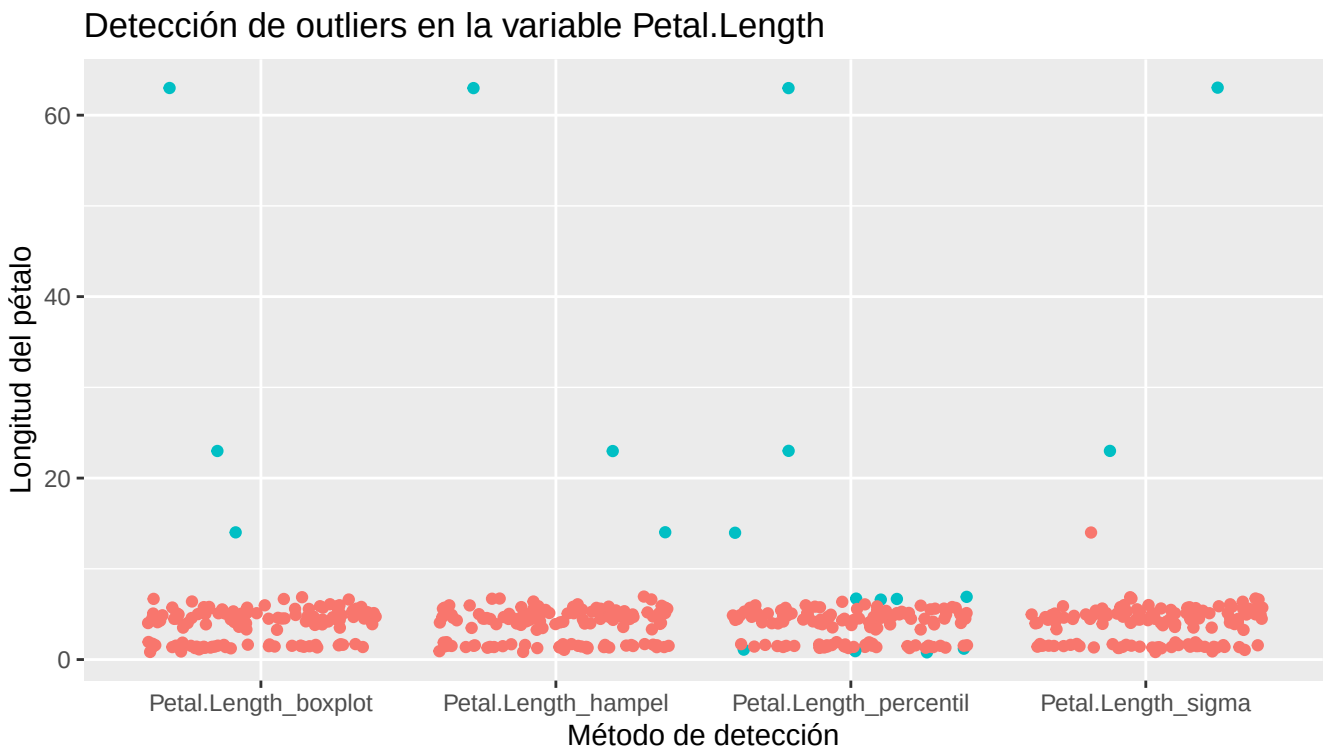
Ejercicio 8: Representación gráfica

Representa gráficamente los datos de `Petal.Length` (distribución de puntos en el eje y) para todas las reglas de detección implementadas (repartidas en el eje x) mostrando los valores identificados como típicos y los identificados como outliers en diferente color. Usa la función `pivot_longer` para poder agrupar por reglas y colorear los outliers fácilmente.

Nota: usa en `geom_point` con la opción `position='jitter'` para mejorar la legibilidad del gráfico.

El procedimiento seguido es bastante similar al del ejercicio anterior, solo que aqui consideramos todos los valores a excepción de los `NA`. Tras filtrar con `filter()`, graficamos el conjunto de valores clasificando lo considerado por cada método como valor atípico con un color distinto al utilizado para el resto de valores.

```
dirty_iris %>%
  select(Petal.Length, starts_with("Petal.Length_")) %>%
  pivot_longer(
    cols = starts_with("Petal.Length_"),
    names_to = "Metodo",
    values_to = "Outlier"
  ) %>%
  filter(!is.na(Outlier)) %>%
  ggplot(aes(x = Metodo, y = Petal.Length, color = Outlier)) +
  geom_point(position = "jitter") +
  labs(
    title = "Detección de outliers en la variable Petal.Length",
    x = "Método de detección",
    y = "Longitud del pétalo",
    color = "Outlier"
  )
```




```
ggsave("./results/outliers_petal_length.png")
```

La gráfica generada complementa el análisis del ejercicio anterior. Lo que más destaca es el criterio utilizado por el método de percentiles, que refleja como considera atípicos los valores que están situados por debajo del percentil 5 y por encima del 95. Por ello, identifica también valores que no están especialmente alejados del resto, siendo el método que más outliers detecta al forzar que un 10% del conjunto de datos se considere atípico.

En la regla de las 3 sigma vuelve a mostrarse menos agresiva con outliers, ya que considera al punto con longitud de petalo igual a 14 como típico cuando el resto no lo hace.

Imputación de datos

Como se ha visto, con *missing data* nos referimos a una observación (o conjunto de observaciones) cuyo valor no es conocido.

Vamos a ver a continuación ejemplos de cómo imputar valores realistas a esas observaciones perdidas.

Ejercicio 9: Imputación de datos perdidos con estadísticos básicos

La opción más sencilla es imputar con un valor estimado de los valores conocidos (por ejemplo, con la media, la mediana, o la moda).

Se puede realizar una imputación generalizada (usando todos los valores de la variable) o una imputación a partir de casos similares (la estimación se realiza teniendo en cuenta otra variable):

- Imputa los missing data en `Petal.Length` empleando la media. Nota: Puedes reemplazar los valores vacios usando `mutate` con `ifelse` creando una nueva variable `Petal.Length1`.

En primer lugar, imputamos los valores perdidos de `Petal.Length` utilizando la media de todos los valores conocidos de la variable. No obstante, la media va a estar condicionada por los valores atípicos que aún no han sido eliminados del dataset sucio.

Para ello, utilizamos `mutate()` junto con `ifelse()`, de forma que, cuando encontramos un `NA`, lo sustituimos por la media calculada con `mean()`. Además, utilizamos `na.rm = TRUE` para ignorar los valores perdidos durante el cálculo, evitando obtener resultados iguales a `NA`.

Finalmente, guardamos el resultado en una nueva columna, `Petal.Length1`, pero manteniendo la original.

```
dirty_iris <- dirty_iris %>% mutate(Petal.Length1 = ifelse(is.na(Petal.Length), mean(Petal.Length, na.rm

sum(is.na(dirty_iris$Petal.Length))
```

```
[1] 20
```

```
sum(is.na(dirty_iris$Petal.Length1))
```

```
[1] 0
```

Verificamos que la imputación es correcta mediante `sum(is.na())`, que contabiliza el numero de muestras con valor `NA`.

```
sum(dirty_iris$Petal.Length1 == mean(dirty_iris$Petal.Length, na.rm = TRUE))
```

```
[1] 20
```

En este caso, se puede ver que al numero total de imputados se les ha asignado el valor de la media general.

- Imputa los missing data en `Petal.Length` empleando en esta ocasión la media para cada tipo de `Species` (usar`group_by` y `mutate`) creando una nueva variable `Petal.Length2`.

En este caso, en vez de utilizar la media, calculamos una media diferente para cada especie.

Para ello, como se indica en el enunciadd, utilizamos `group_by(Species)` para agrupar los registros según su especie y después aplicamos `mutate()` e `ifelse()` para sustituir los valores perdidos por la media. De esta manera, no se están mezclando todos los valores y, por tanto, las medias obtenidas son más representativas.

Finalmente, añadimos `ungroup()` para evitar que operaciones que tengamos que hacer en el siguiente ejerccio se apliquen directamente sobre los grupos que se han creado para esta actualizacion del dataset.

```
dirty_iris <- dirty_iris %>% group_by(Species) %>% mutate(Petal.Length2 = ifelse(is.na(Petal.Length), m

sum(is.na(dirty_iris$Petal.Length))
```

```
[1] 20
```

```
sum(is.na(dirty_iris$Petal.Length2))
```

```
[1] 0
```

En este caso, si comprobamos si los valores que se han imputado tienen el mismo valor que la media global, el resultado es **NINGUNO** ya que se esta utilizando la media de cada especie.

```
sum(dirty_iris$Petal.Length1 == mean(dirty_iris$Petal.Length, na.rm = TRUE))
```

```
[1] 20
```

Ejercicio 10: Representación gráfica

Vamos a comparar ahora los dos métodos de imputación gráficamente eliminando los outliers para una mejor visualización.

- Recalcula las reglas de identificación de outliers (Ejercicio 6) para la nueva variable `Petal.Length2`.

- Representa los dos tipos de imputación empleados en un gráfico de dispersión (`Petal.Length1` vs. `Petal.Length2`) eliminando los outliers identificados con la regla `reglapercentil` y poniendo en diferente color los datos originales y los imputados.